MÁSTER EN ANÁLISIS DE MALWARE Y REVERSING
MÓDULO 7. DEBUG Y ANÁLISIS DEL SISTEMA OPERATIVO (LINUX)
MODO USUARIO Y MODO KERNEL

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

INDICE DE CONTENIDOS

1. Aviso para navegantes: 3

2. Introducción 3

3. Tipos de núcleos 4

4. Módulos del kernel 6
   Implicaciones de seguridad 7

5. El Kernel de Linux 8
   5.1. Kernel de Linux: Consumo de funcionalidades del kernel 9
   5.2. Kernel de Linux: Sistema de ficheros 10
   5.3. Kernel de Linux: Gestión de memoria 11
   5.4. Kernel de Linux: Gestión de procesos 13

6. Estado de ejecución de una aplicación y depuración 15
   6.1. Herramientas útiles en la monitorización de aplicaciones 16
   6.2. Limites de recursos que afectan a cada aplicación 20

7. Información generada por el kernel 20
   7.1. Depuración del kernel 20
   7.2. Métodos alternativos de revisión de la actividad del kernel 22
      7.2.1. System Crash Dumps o core dumps. 22
      7.2.2. Análisis de ficheros de paginación. 25
      7.2.3. System Hang (Congelados del sistema) 28
      7.2.4. El directorio virtual /proc 28
      7.2.5. Añadir printk's al kernel 28

8. Módulos PAM (Pluggable Authentication Modules) 28
   8.1. ¿Qué es un módulo PAM? 28
   8.2. Algunas de las características que ofrece PAM 29
   8.3. Configuración de los módulos PAM 29

Bibliografía 30

1. AVISO PARA NAVEGANTES:

- En este modulo nos centraremos en el kernel Linux y en el sistema operativo Ubuntu como ejemplo de su implementación
- Antes de comenzar, es necesario tener claros los siguientes términos:
    - UNIX
    - LINUX
    - GNU

2. INTRODUCCIÓN

Antes de nada, es fundamental conocer la diferencia entre los dos posibles modos de ejecución del procesador: **Modo kernel o núcleo** (normalmente asociado al sistema operativo) y **modo usuario** (normalmente asociado a los programas de usuario).

La existencia de un modo privilegiado como el modo kernel, permite evitar la ejecución por parte del usuario a instrucciones que rigen el funcionamiento del sistema operativo y que interactúan directamente con el hardware, como por ejemplo realizar cambios en el manejador de instrucciones de reloj, la forma en la que se produce la comunicación de un dispositivo E/S o incluso acceder o alterar la codificación del SO. De este modo, se evitan usos indebidos del SO y del hardware que pudieran derivar en comportamientos inesperados de la máquina o que corrompan su funcionamiento.

Algunas de las operaciones reservadas al modo kernel son: la lectura/escritura/modificación de los registros de control, instrucciones primitivas de E/S e instrucciones relacionadas con la memoria, señales, planificación, interrupciones, controladores de dispositivos, etc.

Todas las operaciones, funcionalidades y servicios que requieren un modo privilegiado de ejecución y, en definitiva, las más importantes, se incluyen en el núcleo del sistema operativo. Esta parte del SO, es relativamente pequeña, sin embargo es la más utilizada, por lo que suele residir en memoria de forma continua.

Lista de funciones básicas del kernel:

- **Gestión de procesos**
    - Creación y terminación de procesos
    - Planificación y activación de procesos

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

        - Intercambio de procesos
        - Sincronización de procesos y soporte para la comunicación entre procesos
        - Gestión de los bloques de control de proceso
- **Gestión de memoria**
        - Reserva de espacio de direcciones para los procesos
        - *Swapping*
        - Gestión y páginas de segmentos
- **Gestión E/S**
        - Gestión de *buffers*
        - Reserva de canales de E/S y de dispositivos de los procesos
- **Funciones de soporte**
        - Gestión de interrupciones
        - Auditoría
        - Monitorización

El motivo de que existan distintos modos de ejecución no es otro que proteger al sistema operativo y tablas clave del sistema de la interferencia de programas de usuario. En modo núcleo, el software tiene control completo del procesador y de sus instrucciones, registros y memoria.

Llegados a este punto, nos surgen dos cuestiones:

1. ¿Cómo conoce el procesador en qué modo se está ejecutando?
2. ¿Cómo puede modificarse este modo?

La respuesta a la primera pregunta: Existe un bit en la palabra de estado de programa (PSW) que indica el modo de ejecución. Este bit altera su valor como respuesta a determinados eventos.

La respuesta a la segunda pregunta: Cuando un usuario realiza una llamada a un servicio del sistema operativo o cuando una interrupción dispara la ejecución de una rutina del sistema operativo, este modo de ejecución se cambia a modo núcleo (alternando el bit anteriormente mencionado) y cuando finaliza la ejecución del servicio, el modo se fija de nuevo a modo usuario.

3. TIPOS DE NÚCLEOS

Existen numerosos tipos de núcleos en base a su organización y funcionamiento, además de su arquitectura. Para cada uno de estos tipos, existen a su vez multitud de implementaciones y soluciones.

Principalmente existen cuatro grandes tipos de núcleos:

- Los **núcleos monolíticos** facilitan abstracciones del hardware subyacente realmente potentes y variadas.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- Los **micronúcleos** (en inglés ***microkernel***) proporcionan un pequeño conjunto de abstracciones simples del *hardware*, y usan las aplicaciones llamadas servidores para ofrecer mayor funcionalidad.
- Los **núcleos híbridos** (*micronúcleos modificados*) son muy parecidos a los micronúcleos puros, excepto porque incluyen código adicional en el espacio de núcleo para que se ejecute más rápidamente.
- Los **exonúcleos** no facilitan ninguna abstracción, pero permiten el uso de bibliotecas que proporcionan mayor funcionalidad gracias al acceso directo o casi directo al hardware.

En el presente módulo, nos centraremos en el kernel de Linux (https://es.wikipedia.org/wiki/N%C3%BAcleo_(inform%C3%A1tica) y en concreto, usaremos el sistema operativo Ubuntu como ejemplo de implementación del mismo.

El kernel de Linux (monolítico) (https://es.wikipedia.org/wiki/N%C3%BAcleo_Linux) es una adaptación del kernel de UNIX (micronúcleo).

El acceso al kernel de Linux, se realiza mediante peticiones a través de una biblioteca (biblioteca del lenguaje C) que permite el acceso a todas las funciones que ofrece el kernel. En un sistema con núcleo monolítico como Linux la biblioteca de lenguaje C consiste en una abstracción de acceso al núcleo. Algunas bibliotecas como la biblioteca de GNU (también conocida como glibc) proveen funcionalidad adicional para facilitar la vida del programador y usuario o mejorar el rendimiento de los programas.

El consumo de funciones del núcleo se hace a través de llamadas al sistema.

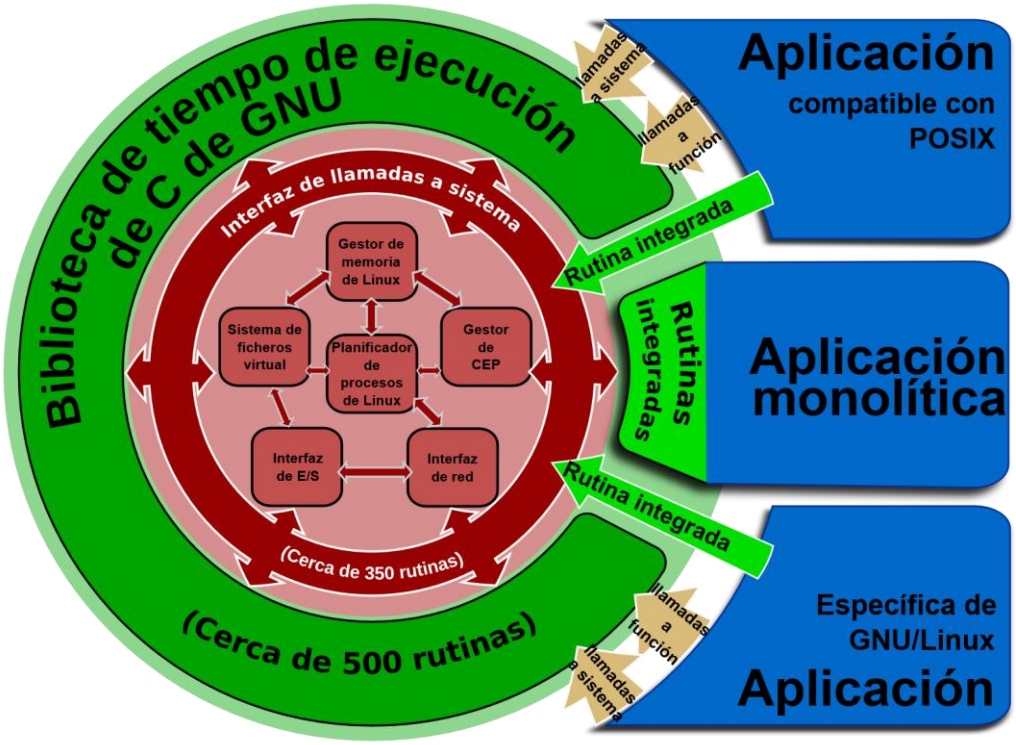


Imagen de Raul Vidal - Trabajo propio, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=62260998

La biblioteca de C de GNU provee funcionalidad que aísla a las aplicaciones del núcleo (Linux en esta imagen). Para utilizar funcionalidad específica de Linux se emplea la rutina syscall() de la biblioteca de C de GNU. O bien una implementación propia de la llamada a sistema en ensamblador. Si una aplicación utiliza la interfaz de llamadas a sistema de Linux con implementaciones propias en exclusiva, entonces dicha aplicación se considera monolítica.

4. MÓDULOS DEL KERNEL

El kernel distribuye sus funcionalidades en una serie de modulos, secciones software que ejecutan tareas relacionadas. Gracias a la posibilidad de cargar módulos de la mayoría de núcleos, es posible extender su funcionalidad mediante módulos cargables de núcleo https://es.wikipedia.org/wiki/M%C3%B3dulo_de_n%C3%BAcleo. Así, partiendo de un núcleo en ejecución (núcleo base) es posible dar soporte a nuevo hardware y sistemas de archivos, así como incorporar nuevas llamadas al sistema.

Es importante entender la diferencia entre módulos y aplicaciones. Si bien las aplicaciones de tamaño pequeño a mediano están diseñadas para realizar una única tarea de inicio a fin, un módulo del núcleo es cargado con el objetivo de servir en cualquier futuro momento en el que sea necesario. De hecho, por lo general el estilo de programación de los módulos del kernel guarda bastante similitud con la programación dirigida por eventos. Otra diferencia fundamental es la tolerancia a fallos, ya que, si una aplicación falla o aborta errores, suele ser inofensivo para el sistema operativo, mientras que el fallo de un módulo, producirá con bastante probabilidad un fallo en el kernel que podría dejar el sistema en un estado inconsistente e incluso inutilizable. Por supuesto, la diferencia principal es que los módulos se ejecutan en espacio kernel mientras que las aplicaciones (sea cual sea su tamaño o función) se ejecutan en espacio usuario.

Los módulos cargados en el sistema se especificarán en el fichero /etc/modules.conf y podrán ser consultados a través del siguiente comando de consola:

/sbin/lsmod

Implicaciones de seguridad

Si bien los módulos cargables consisten en un método conveniente para modificar el núcleo en ejecución, esto podría llegar a ser abusado por un atacante en un sistema comprometido, con el fin de prevenir la detección de procesos o archivos maliciosos, permitiéndole mantener control sobre el sistema o robar información privada.

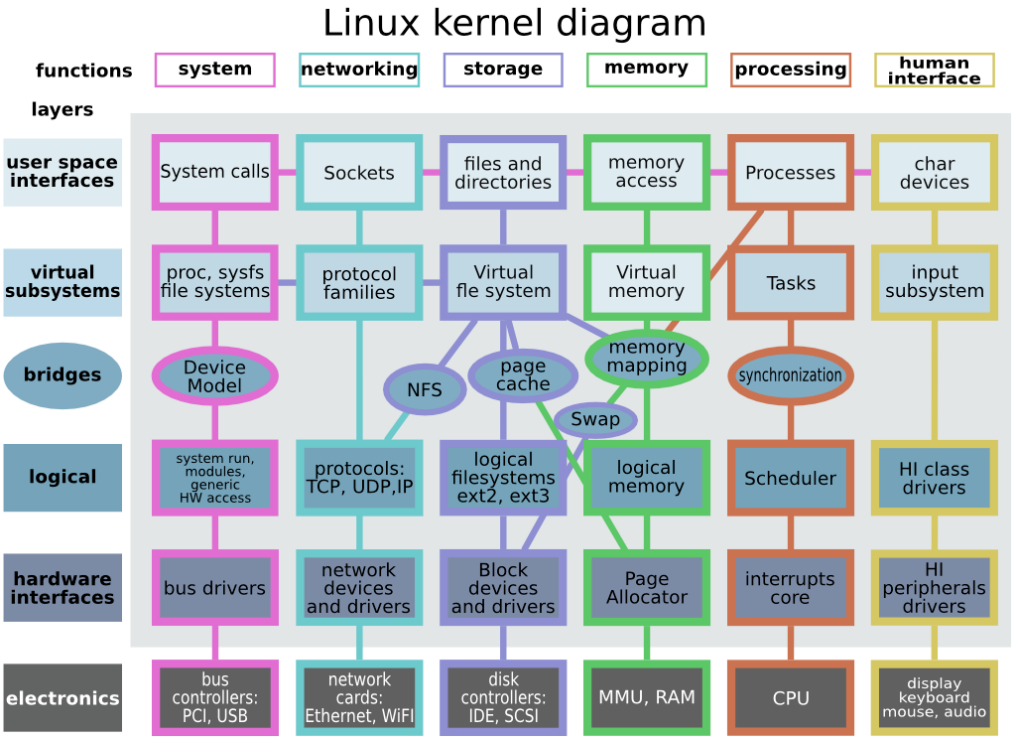
En el caso de Linux, la mayoría de los controles de seguridad del sistema se refuerzan a través del núcleo. Si éste tiene problemas de seguridad, el sistema completo será vulnerable. En la distribución oficial del núcleo, solo un usuario autorizado, conocido como el superusuario, puede cargar módulos.[1]

A su vez, los desarrolladores de módulos cargables deberían ser cuidadosos de no introducir errores de seguridad en los módulos que desarrollan. El lenguaje de programación C, utilizado en la mayoría de Linux, hace que el introducir errores sea fácil y muchos problemas de seguridad suelen deberse a que los desarrolladores olvidan controlar cuánta información es almacenada en una porción asignada de memoria (buffer), derivando en errores de sobrescritura de memoria[1] que podrían ser utilizados por un usuario malicioso para alterar la memoria del núcleo y sus módulos.

En los sistemas operativos que trabajan bajo la norma POSIX o similares, algunas llamadas al sistema muy utilizadas son open, read, write, close, wait, exec, fork, exit y kill.

5. EL KERNEL DE LINUX

El kernel de Linux fue inicialmente creado por Linux Torvalds en 1991 como un sistema operativo basado en Unix. El siguiente diagrama muestra una estructura de los diferentes elementos que nos podemos encontrar en él núcleo, también es posible consultar la estructura del kernel en este mapa interactivo.



Los componentes del núcleo de Linux pueden ser clasificados a alto nivel según la función que desempeñan dentro del mismo:

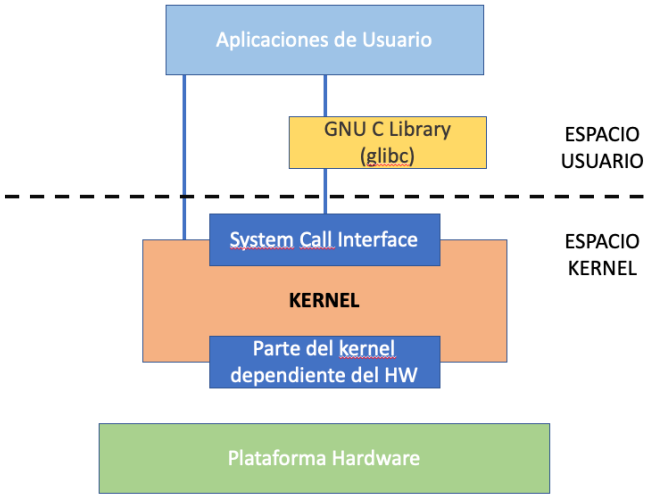
- **Interfaz humana**: En él nos encontramos los drivers y dispositivos de interacción con el usuario (ratón, teclado, altavoces, etc.),
- **Procesamiento**: Componentes relacionados con la gestión de procesos, tareas, sincronización, planificación de procesos e interrupciones.
- **Memoria**: Se encargan de la virtualización, direccionamiento, intercambio de memoria a disco, asignación de páginas, etc.
- **Almacenamiento**: Interacción con los dispositivos de almacenamiento, gestión de sistemas de ficheros, implementación de sistemas de ficheros virtuales, cacheado de páginas, etc.
- **Redes**: Gestión de conexiones, familias de protocolos, interacción con los dispositivos físicos, etc.

- **Sistema**: Se encargan de las llamadas al sistema, los sistemas de ficheros virtuales de diagnostico del kernel, el modelo de dispositivo, etc.

5.1. KERNEL DE LINUX: CONSUMO DE FUNCIONALIDADES DEL KERNEL

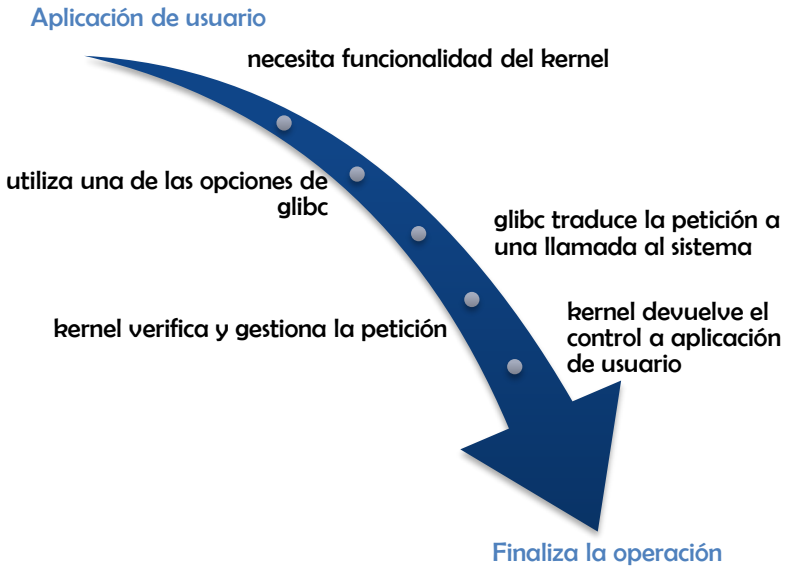
¿Cómo puede un proceso de un usuario acceder a la funcionalidad del kernel si se ejecuta en espacio de usuario?

Cuando un programa de usuario necesita una funcionalidad del kernel, este solicita la funcionalidad que desea realizar al kernel, cediéndole el control. Para realizar esta acción, los programas utilizan la librería glibc que implementa la API de interacción kernel-usuario (System Call Interface) o implementan su propia llamada al kernel.



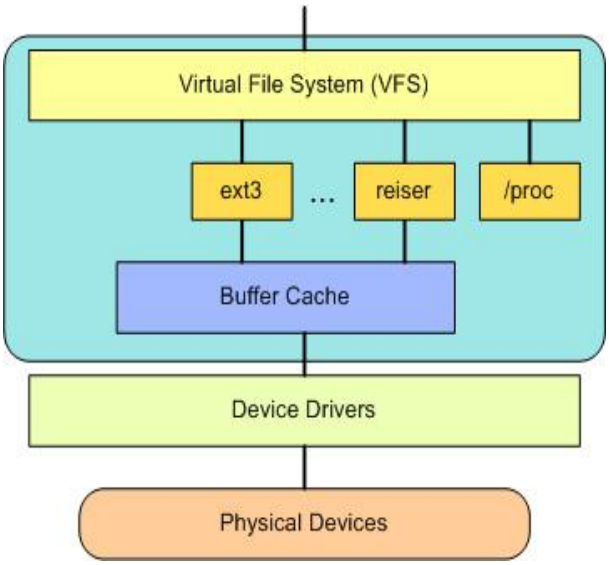
Cuando el kernel toma el control, se dice que la ejecución pasa a modo kernel, teniendo acceso a toda la funcionalidad del núcleo.

Una vez el kernel ha finalizado con la petición del usuario, la ejecución entra de nuevo en modo restringido (modo usuario) y el control es devuelto al proceso del usuario.



5.2. KERNEL DE LINUX: SISTEMA DE FICHEROS

El sistema de ficheros implementa numerosos mecanismos que permiten al sistema operativo ir progresivamente **abstrayéndose de la implementación** a bajo nivel de cada dispositivo físico. Esto se consigue con la **estructura de capas** que incorpora, siendo la primera el **sistema de ficheros virtual (VFS).** El resto de funciones del kernel interactúan con el VFS de forma unificada e independientemente del fabricante, familia, modelo e incluso estructura lógica del dispositivo que se esté utilizando. Esto permite al kernel abstraerse de todo ese proceso y simplificar la actuación.



Un paso mas abajo nos encontramos el **sistema de ficheros** ("real"). Ejemplos de sistemas de ficheros son Ext2, ext4, /proc. El sistema de ficheros se encarga de la **gestión lógica** de **cómo los datos son almacenados y recuperados**. En esta capa, en parte aun es independiente del dispositivo hardware, aunque la interacción es muy estrecha. Entre las principales tareas del sistema de ficheros se encuentran la gestión del espacio libre y utilizado, la gestión de nombres de ficheros, la interacción con el dispositivo físico a través de los drivers de forma óptima (mediante el uso de buffers de escritura y lectura, reducción de fragmentación, etc.), la gestión de acceso a los ficheros, así como su organización en directorios, etc.

En última instancia, nos encontramos **los drivers y el componente hardware**, encargado de llevar a cabo las operaciones de escritura y lectura sobre el dispositivo.

Enlaces interesantes...

https://www.win.tue.nl/~aeb/linux/lk/lk-8.html
http://www.tldp.org/LDP/intro-linux/html/sect_03_01.html
http://www.tldp.org/LDP/tlk/fs/filesystem.html#ext2fs-figure

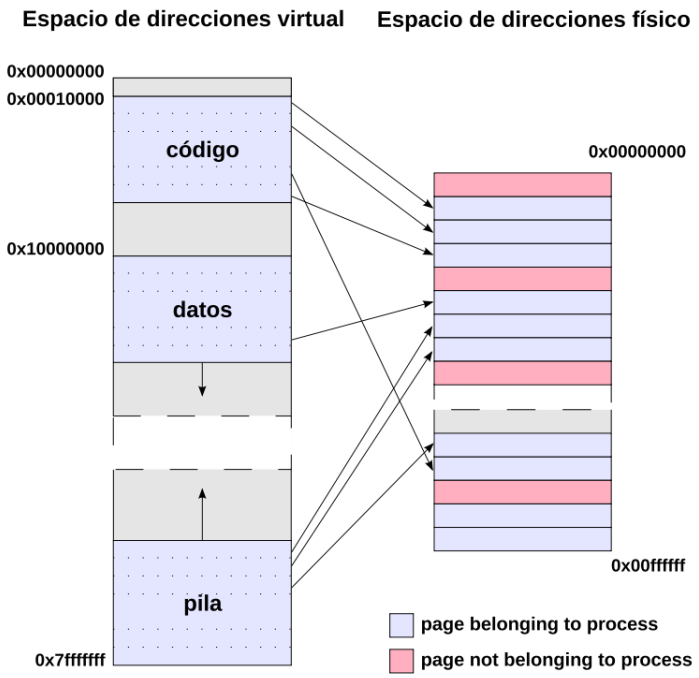
5.3. KERNEL DE LINUX: GESTIÓN DE MEMORIA

El sistema de memoria es uno de los más complejos que podemos encontrar dentro de un computador, implicando componentes hardware y software que incluyen deferentes mecanismos de protección. En primer lugar, cabe destacar que la memoria se estructura en páginas que dependiendo del sistema pueden variar su tamaño. Adicionalmente, cabe destacar

que existen principalmente dos componentes que hacen uso de la misma. Estos componentes son la CPU y los dispositivos de entrada/salida (véase discos duros), los cuales pueden interactuar en determinadas situaciones directamente con la memoria sin la necesidad de utilizar la CPU.

Por otro lado, se disponen de diferentes niveles de caches de páginas cuya velocidad de acceso está inversamente relacionada con su tamaño (debido al precio) para finalmente confluir en la memoria principal.
El sistema de gestión de memoria cuenta con mecanismos de protección y optimización que se gestionan a mitad de camino entre los componentes hardware, drivers y el propio sistema operativo. Algunos de ellos son los siguientes:

- **Virtualización de memoria**: Permiten desacoplar las direcciones que un proceso ve con respecto a las direcciones de memoria reales.

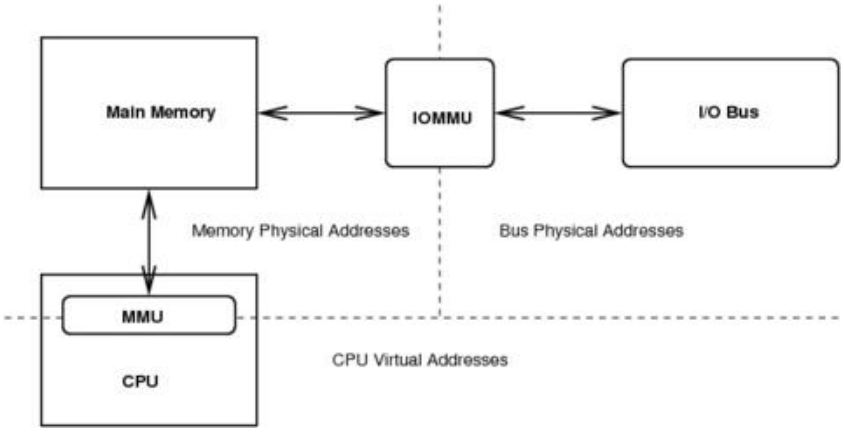


- **Data Execution Prevention (DEP)**: Protección de ejecución en zonas de memoria dedicadas a datos.

- **Aleatorización en la disposición del espacio de direcciones (ASLR)**: Distribución aleatoria de las posiciones de memoria que contienen secciones críticas del proceso. El

término "ASLR" fue acuñado por primera vez en el proyecto PaX, un parche del núcleo de Linux diversas protecciones a las páginas de memoria, publicando su diseño y primera implementación en julio de 2001.

Gracias a la virtualización se consigue un uso mucho más eficiente de la memoria, evitando además mediante el uso de mecanismos como DEP o ASLR que procesos no deseados o dispositivos **defectuosos, maliciosos** o mal desarrollado tengan acceso a regiones críticas de la **memori**a. Además, la virtualización da la posibilidad al hardware de acceder directamente a la memoria a través de direcciones virtuales controladas por mecanismos como las unidades IOMMU (Input–Output Memory Management Unit) sin necesidad de hacerlo a través de la CPU, que a su vez usará direcciones virtualizadas a través de la MMU (Unidad de Gestión de Memoria Principal). Una de las principales funciones de seguridad de la MMU es gestionar el mapeo de direcciones virtuales sobre las direcciones físicas, que permite al sistema operativo aislar a unos procesos de otros "haciéndoles creer" que se encuentran solos en el sistema.



Enlaces interesantes...

https://www.tldp.org/LDP/tlk/mm/memory.html
https://www.linuxjournal.com/article/7104

5.4. KERNEL DE LINUX: GESTIÓN DE PROCESOS

Un **proceso** en Linux es una instancia de un programa que está siendo ejecutada en el sistema operativo. Cada proceso siempre tiene un proceso padre, a excepción del proceso **Init**, el cual

es el primero en ejecutarse cuando se arranca el sistema operativo y a su vez, cada proceso puede tener procesos hijos.

Entre los elementos de un proceso nos encontramos los siguientes:

- **Control de estado**: Cada proceso tiene asociado un estado el cual oscila entre los valores "Running", Waiting", "Stopped", "Zombie", según su estado de ejecución.
    - **Running**: Los procesos que se encuentran en este estado están en ejecución o listos para ser ejecutados.
    - **Waiting**: El proceso está esperando hasta la ocurrencia de un determinado evento o la disponibilidad de un cierto recurso.
    - **Stopped**: El proceso está detenido, normalmente por la recepción de una señal de stop.
    - **Zombie**: Los procesos en estado zombie, son aquellos procesos que han sido detenidos, pero aún se mantiene información sobre su ejecución en los sistemas de gestión de procesos. Normalmente esto suele ocurrir para permitir a otro proceso obtener información del estado de finalización del proceso que quedó en estado zombie.
- **Información de planificación**: Información utilizada por el planificador de procesos para decidir que proceso debe ser ejecutado en cada instante de tiempo.
- **Identificadores**: Cada proceso en el sistema operativo cuenta con un identificador único (**PID**) que le permite ser referenciado por el resto de los componentes del kernel. Adicionalmente, cada proceso cuenta con un identificador del usuario y grupo al que pertenece su ejecución, los cuales son utilizados para controlar el acceso del proceso a los ficheros y dispositivos en el sistema. El número máximo de PID que pueden ser asignados se encuentra definido en el pseudofichero **/proc/sys/kernel/pid_max**

- **Enlaces**: En cada proceso, podemos encontrar un enlace a su proceso padre (PPID), a cada uno de sus procesos hijos, incluso a sus procesos hermanos. Esto permite ver la familia de un proceso de forma sencilla utilizando por ejemplo el comando pstree.
- **Temporizadores**: En un proceso podemos encontrar el temporizador que refleja el tiempo que ha pasado desde su creación, así como el tiempo que ha consumido de uso de CPU. Adicionalmente podemos encontrar algunos temporizadores que los procesos pueden definir para que el kernel los notifique cuando expire.
- **Tabla de ficheros abiertos**: Los procesos pueden utilizar, siempre que dispongan de permisos de acceso, los ficheros que necesiten para su funcionamiento. Por ello, en cada proceso se almacena información sobre cada uno de los ficheros que están siendo utilizados en cada momento. Para ello, existe una tabla con punteros a los descriptores de ficheros abiertos por el proceso.

- **Mapeo de memoria virtual**: En esta estructura se almacena la información necesaria para poder resolver el espacio de direcciones del proceso, es decir, saber traducir entre direcciones de memorias virtuales (para este proceso) y direcciones de memoria física (reales).
- **Contexto del proceso**: Cuando un proceso deja de utilizar la CPU, ya sea por acción del planificador o porque está a la espera de un recurso, el kernel almacena el estado en el que quedó su ejecución en esta estructura de forma que posteriormente pueda ser reanudada por el mismo punto.

Adicionalmente, existe un concepto de proceso ligero o hebra. Una **hebra** es un proceso que comparte parte de su estructura con su proceso padre y sus hebras hermanas, de esta forma se optimizan los cambios de contexto dentro del sistema operativo.

Los procesos, compiten por el uso del procesador, el cual es un recurso limitado. Por ejemplo, si en un computador existen 4 procesadores, físicamente solo podrán estar en ejecución 4 procesos simultáneamente, lo que hará al resto de procesos esperar. Para repartir el tiempo de uso del procesador, en el kernel existe la figura del planificador de procesos o planificador de tareas. El planificador de tareas, evalúa el estado de cada proceso, el tiempo de ejecución del mismo, el tiempo que ha utilizado la CPU, y una serie de parámetros más en concordancia con el resto de procesos para decidir en cada momento que procesos pueden utilizar la CPU. Cuando un proceso deja la CPU, el planificador realiza una acción llamada cambio de contexto, la cual consiste en almacenar el estado de los registros, pilas y demás componentes compartidos para permitir retomar el trabajo por el mismo punto cuando el proceso vuelva a disponer de tiempo de acceso a la CPU.

6. ESTADO DE EJECUCIÓN DE UNA APLICACIÓN Y DEPURACIÓN

Es importante conocer el estado de ejecución de una aplicación, que llamadas al núcleo ejecuta (si lo hace) y con qué programas del sistema operativo interacciona. A su vez, debemos tener en cuenta cuales son los principales programas del sistema operativo y cual es la huella típica que deja su ejecución, así como cuales son los mecanismos que podemos ejecutar para su depuración, así como conocer su estado de ejecución.

6.1. HERRAMIENTAS ÚTILES EN LA MONITORIZACIÓN DE APLICACIONES

- **pstree**

Permite desplegar un árbol de procesos e investigar qué cantidad de procesos y hebras genera cada proceso o grupo de procesos. Permite añadir información extra a cada proceso a través de opciones como por ejemplo -g (equivalente a –show-pgids) que permite listar junto a cada proceso el group id del grupo de procesos al que pertenece) o -p (equivalente a –show-pids), que muestra cual es el PID de cada uno de los procesos mostrados en el árbol.

Mediante la opción -T (equivalente a --hide-threads), es posible ocultar del árbol aquellos registros que pertenezcan a hebras, dejando así únicamente aquellas pertenecientes a procesos. Por el contrario, usando la opción -t (equivalente a --thread-names) es posible extender la información de los registros que representan hebras, mostrando además el nombre de estas hebras.

Por último, una opción fundamental a la hora de depurar aplicaciones usando pstree es -a (equivalente a --arguments), que muestra junto a cada proceso o hebra el comando y argumentos mediante los que fueron lanzados.

```
Last login: Sun Feb  9 17:14:58 2020 from 172.16.60.1
admin1@mru8:~$ pstree
systemd─┬─VGAuthService
        ├─accounts-daemon───2*[{accounts-daemon}]
        ├─atd
        ├─cron
        ├─dbus-daemon
        ├─login───bash
        ├─lvmetad
        ├─lxcfs───2*[{lxcfs}]
        ├─networkd-dispat───{networkd-dispat}
        ├─polkitd───2*[{polkitd}]
        ├─rsyslogd───3*[{rsyslogd}]
        ├─snapd───8*[{snapd}]
        ├─sshd─┬─sshd───sshd───bash───python3───5*[{python3}]
        │      └─sshd───sshd───bash───pstree
        ├─systemd───(sd-pam)
        ├─systemd-journal
        ├─systemd-logind
        ├─systemd-network
        ├─systemd-resolve
        ├─systemd-timesyn───{systemd-timesyn}
        ├─systemd-udevd
        ├─unattended-upgr───{unattended-upgr}
        └─vmtoolsd───{vmtoolsd}
```

Junto a herramientas como top o htop permite obtener una amplia visión del uso de recursos que se está haciendo de la máquina a nivel proceso.

Imaginemos por ejemplo el caso en el que uno de los procesos de uno de nuestros entornos de producción, está generando un número inusitado de hebras. Por ejemplo, el driver oficial para Java de bases de datos MongoDB, puede comenzar a lanzar de forma indefinida hebras instancia del limpiador de cursores de la BD "Mongo Cleaner" si durante la ejecución normal de una aplicación, no se están cerrando correctamente las conexiones a base de datos o si no se están instanciando correctamente los clientes. En el siguiente enlace vemos como un usuario muestra cómo el proceso Java que está ejecutando, levanta más de 20000 hebras "Mongo Cleaner". Unas de las primeras principales que puede generar este comportamiento, es que se alcance el número máximo de procesos que el sistema operativo permite para el usuario, impidiendo la ejecución de nuevos procesos asociados a este usuario, lo que implica, por ejemplo, que no se podría hacer login vía SSH a la máquina usando ese usuario. Afortunadamente, el usuario root no tiene límite de procesos, aunque no es una opción recomendable permitir la autenticación mediante este usuario. Recordemos que es posible obtener el número máximo de procesos que admite nuestra máquina consultando la información del fichero `/proc/sys/kernel/pid_max`. En el caso de hebras, es posible hacer lo mismo usando los comandos:

En el supuesto anterior, las hebras lanzadas prácticamente no tienen consumo de CPU ni de RAM, por lo que sería complicado mediante comandos como htop (aunque contiene un visor de árboles de procesos) detectar cuál es el problema hasta que no ocurren las primeras consecuencias como por ejemplo fallas en la ejecución de procesos. En este caso, una vez detectado el problema principal (por ejemplo, al intentar hacer login.

- **lsof**

La herramienta de monitorización **lsof** (list open files) permite mostrar todos los archivos de disco que mantienen abiertos los procesos, incluyendo los sockets de red abiertos y pipeline de procesos entre otros.

El comando **lsof** brinda al usuario información detallada acerca del contexto donde se utiliza el archivo. Entre la información se encuentra:

- nombre e id del proceso,
- nombre o id del usuario que lanzó dicho proceso,
- El descriptor de archivo junto con el modo de apertura (lectura,escritura,etc..),
- El tipo de nodo asociado al archivo: Tubería (FIFO), Socket IPV4, Socket IPv6, Directorio, Bloque de Dispositivo,etc..
- La ruta absoluta del archivo o bien información relativa según el tipo,
- El contexto de seguridad SELinux

Si no se indica ningún parámetro **lsof** enumera todos los archivos abiertos en ese momento, que normalmente suelen ser bastantes.

Algunos de los parámetros mas interesantes de lsof son:

-a

Permite que las opciones (parámetros) sean requisitos para listar. Útil cuando se "filtra" usando **-c**, **-g**, **-u** o **-p**.

-c *CADENA*

Lista los archivos de los procesos que comiencen con *CADENA*

-d *FDs*

Determina que se deben excluir el o los Descriptores de Archivos (FD) del listado. Los valores se separan por comas.

+d *DIRECTORIO*

Lista los procesos que utilicen el directorio o bien alguno de los archivos que contenga.

+D *DIRECTORIO*

idem al anterior pero recursivo: Incluye a los subdirectorios y archivos que estos contengan

-i

Lista los sockets abiertos.

-g GID

Lista los archivos lanzados por el GID (ID de Grupo) indicado.

-n

No resuelve los nombres de dominio.

-p *PID*

Lista los archivos abiertos por el proceso con el PID (ID de Proceso) indicado.

-P

No resuelve los nombres de los puertos (conexiones).

[+|-]r [*N* [m*CADENA*]]

Imprime en pantalla el resultado cada *N* segundos, si no se especifica se ejecuta cada 15. Si se utiliza el prefijo '-' se repite esta operación hasta que el usuario lo cancele (ctrl+C), en cambio si se utiliza el prefijo '+', el proceso termina cuando no haya archivos listados. Adicionalmente entre cada actualización es posible indicarle cual es la cadena de texto que separa cada resultado, a través m*CADENA*. En caso de no indicarle ninguna cadena, por default se utiliza '=======‘.

-u [*usuario*|*UID*]

Lista los archivos abiertos por el usuario o bien UID (ID de Usuario).

-U

Lista los sockets UNIX (IPC sockets).

A modo de ejemplo, imaginemos que desplegando un servidor de aplicaciones web que escucha el puerto 443 (HTTPS) en una de las máquinas con las que trabajamos, nos encontramos con que no puede ser lanzado porque ya hay un proceso escuchando en ese mismo puerto. En este caso, podríamos usar el siguiente comando para averiguar de qué proceso se trata:

lsof -i | grep :443

Una vez obtenido el proceso que está escuchando ese puerto, será posible comprobar a qué aplicación corresponde (si no queda claro con la información que ya arroja lsof) a través de comandos como ps aux [PID], que nos permitirán conocer entre otros, cual fue el comando de lanzamiento del proceso.

6.2. LIMITES DE RECURSOS QUE AFECTAN A CADA APLICACIÓN

Para conocer el estado en el que se ejecutan las aplicaciones de un sistema operativo, será también necesario conocer el contexto en el que estas se ejecutan, así como las restricciones bajo las que trabajan, ya sea impuestas por el propio sistema operativo, los distintos usuarios o incluso por otras aplicaciones.
Sean aplicadas directamente o como resultado indirecto (y probablemente indeseado) de una limitación demasiado amplia a otras aplicaciones o recursos (usuarios, grupos, etc), debemos ser capaces de conocer estas limitaciones independientemente de que fuéramos nosotros quienes las establecieran o no mediante el uso de los mecanismos de limitación explicados en la unidad anterior y la huella que estos dejan en el sistema. Así por ejemplo, si previamente se ha restringido el numero máximo de procesos que podrá ejecutar un usuario, es posible que en el futuro algunas aplicaciones comiencen a tener problemas a la hora de lanzar nuevas tareas, por lo que en la fase de depuración, deberíamos comprobar el fichero /etc/security/limits.conf y en concreto, las configuraciones que incluyan el item **nproc**.

Los errores en tiempo de ejecución relacionados con limites del sistema, suelen ser difíciles de detectar e identificar correctamente, por lo que conviene tener siempre en cuenta esta posibilidad en la depuración de errores del sistema operativo y las aplicaciones que se ejecutan sobre él.

7. INFORMACIÓN GENERADA POR EL KERNEL

La depuración estricta del kernel queda fuera del objetivo de esta asignatura debido a la complejidad y amplitud de la materia, así como el tiempo requerido para realizar pruebas con él (solo su compilación puede tomar horas de cómputo de la máquina). Por este motivo, este módulo se centrará en la recolección y análisis de información que genera el kernel utilizando distintas fuentes y técnicas que, si bien no permiten el mismo nivel de detalle que la depuración, pueden ayudarnos a conocer qué operaciones se están utilizando, tiempos de ejecución, posibles estados de error, etc.

7.1. DEPURACIÓN DEL KERNEL

Para la depuración del kernel, es necesario el uso de un depurador. La función del depurador es controlar la actividad del kernel y ofrecer una interfaz de operación al usuario ya sea local o remota que le permita revisar paso a paso las instrucciones del kernel que se van ejecutando en cada momento, así como controlar su ejecución.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Para depurar el kernel de Linux existen varias opciones. Algunas de ellas:

- **KDB** (local)
- **KGDB** (remoto)
- **MDB** (local/remoto)
- **GDB** (Depurador estándar para el compilador GNU). Fue escrito por Richard Stallman en 1986. Permite depurar tanto en remoto como en local además de la pobilidad de usarlo mediante interfaces gráficas diseñadas por terceros.

La ventaja de los kernel que permiten depuración remota es que es posible controlar la depuración desde una máquina distinta a la que la está ejecutando, así como hacerlo desde cualquier parte.

En el Kernel de linux no se incluyó oficialmente ningún depurador del kernel antes de la versión 2.6.26-rc1 ya que Linus Torvalds no quería un depurador del kernel en su creación.

La depuración del kernel, requiere en primer lugar la recompilación del mismo, habilitando la opción de depuración. Una práctica recomendada para evitar un tiempo excesivo de compilación es seleccionar únicamente las opciones y módulos necesarios para nuestro caso.

El uso del depurador del kernel, suele imponer limitaciones sobre el uso de recursos de la máquina, como por ejemplo modo de ejecución del procesador, uso de memoria, virtualización, etc. Así por ejemplo, cuando el depurador KDB es invocado por el usuario, es el único programa en ejecución. Uno de los procesadores es seleccionado como procesador de debug y todos los demás procesadores del sistema se detendrán mediante el envío de una interrupción **IPI** (Interprocessor Interrrupt) a cada uno de los demás procesadores. En caso de que alguno de los procesadores no pueda ser detenido, el depurador mostrará una advertencia.

En el caso de que el depurador haya sido lanzado de forma automática (por ejemplo tras una excepción no controlada del código del kernel), el procesador elegido como procesador de depuración, será habitualmente aquél en el que ha ocurrido la actividad inusual.

Documentación detallada sobre el depurador KDB, sus comandos y su uso práctico durante la depuración:
https://www.ibm.com/support/knowledgecenter/ssw_aix_71/kerneldebugger/kerneldebugger_pdf.pdf

7.2. MÉTODOS ALTERNATIVOS DE REVISIÓN DE LA ACTIVIDAD DEL KERNEL

En los casos en los que no sea necesario un análisis de la actividad del kernel tan pormenorizada como la que ofrece la depuración paso a paso de las instrucciones ejecutadas por el procesador, es posible obtener información de esta actividad mediante el uso de diferentes métodos y herramientas. A continuación, se detallan algunos de los cuales se incluye información más detallada y ejemplos prácticos en la siguiente unidad.

7.2.1. System Crash Dumps o core dumps.

Cuando un sistema colapsa, automáticamente genera una imagen con el estado de la memoria utilizada por cada uno de procesos que se encontraban en ejecución en el procesador en el momento del fallo. Aunque estos volcados de memoria pueden ser cargados y analizados por depuradores como por ejemplo GDB, existe software especializado en la carga, disección y análisis de alto nivel de estos datos. Es importante tener en cuenta que en numerosas ocasiones estas secciones de la memoria pueden tener datos críticos de la máquina como por ejemplo contraseñas, direcciones, usuarios, etc. que se encontraban en memoria en el momento del colapso del sistema.
En el caso concreto de Ubuntu los core dumps con manejados por Apport y suelen alojarse en **/var/crash**/, pero esta funcionalidad está desactivada por defecto para las versiones estables. Sin embargo, puedes ser útil activar esta funcionalidad en algunas situaciones, como por ejemplo cuando por alguna configuración de nuestra máquina o ante un determinado evento esta entra en Kernel Panic. De este modo, será posible será posible analizar el estado de la máquina con el objetivo de encontrar el problema que produce esta situación.

Los pasos para activar los crash dumps del sistema (al menos en ubuntu) son los siguientes:

- Activar apport:
  sudo systemctl enable apport.service o **sudo service apport start**
- Activar la creación de corefiles en la configuración de **ulimit**:
  ulimit -c unlimited
- Verificar que no existen restricciones de escritura de corefiles:
  ulimit -a

```
admin1@mru8:~$ ulimit -a
core file size          (blocks, -c) unlimited
data seg size           (kbytes, -d) unlimited
scheduling priority             (-e) 0
file size               (blocks, -f) unlimited
pending signals                 (-i) 3603
max locked memory       (kbytes, -l) 16384
max memory size         (kbytes, -m) unlimited
open files                      (-n) 1024
pipe size            (512 bytes, -p) 8
POSIX message queues     (bytes, -q) 819200
real-time priority              (-r) 0
stack size              (kbytes, -s) 8192
cpu time               (seconds, -t) unlimited
max user processes              (-u) 3603
virtual memory          (kbytes, -v) unlimited
file locks                      (-x) unlimited
```

Salida del comando ulimit -a donde se aprecia que no existen limitaciones de espacio en los ficheros corefile

- Comprobar que la configuración de generación de corefiles es correcta:

cat /proc/sys/kernel/core_pattern

La salida será similar a esto:

|/usr/share/apport/apport %p %s %c

- En caso de que la configuración no sea correcta, siempre puede modificarse:

sudo sysctl -w kernel.core_pattern=core.%u.%p.%t

Se recomienda reiniciar apport:

sudo service apport restart

La activación de crashdumps o coredumps está desaconsejada salvo en situaciones especiales como por ejemplo si se trata de un proceso de depuración de la misma, debido a la cantidad de información sensible que estos pueden contener (contraseñas, claves privadas, etc.) y que podría ser accesible por cualquiera que tenga acceso de lectura.

El análisis de crash dumps también es especialmente útil en análisis post-mortem en máquinas de terceros o en procesos de pentesting.

Ejemplo: Creación de un programa que fuerce un coredump y lectura del fichero corefile asociado:

1) Creación de un programa que fuerce un error crítico. En este caso, división por cero:

```
admin1@mru8: ~
GNU nano 2.9.3                    crash.c

int main()
{
    return 1/0;
}
```

2) Compilación y ejecución del programa. Como se puede observar en la siguiente imagen, el propio compilador nos avisa (warning) de que la operación puede acarrear un error de división por cero.

```
admin1@mru8:~$ gcc -o crash crash.c
crash.c: In function 'main':
crash.c:3:13: warning: division by zero [-Wdiv-by-zero]
     return 1/0;
             ^
admin1@mru8:~$ ./crash
Floating point exception (core dumped)
```

3) Localizado y lectura del dump asociado.

```
admin1@mru8:~$ sudo cat /var/crash/core.1000.7536.1612129137
 ELF>@@80
OpTpT` haU haU h
!  haU8
 (ELIFCORE
*/home/admin1/crash/home/
admin1/crash/home/admin1/crash/lib/x86_64-linux-gnu/libc-2.27.so/lib/x86_64-linux-gnu/libc-2.27.so/lib/x86_64-lin
ux-gnu/libc-2.27.so/lib/x86_64-linux-gnu/libc-2.27.so/lib/x86_64-linux-gnu/ld-2.27.so/lib/x86_64-linux-gnu/ld-2.2
7.so/lib/x86_64-linux-gnu/ld-2.27.soCORE
/lib64/ld-linux-x86-64.so.2GNUGNUh
"libc.so.6__cxa_finalize__libc_start_mainGLIBC_2.2.5_ITM_
eTable__gmon_start___ITM_registerTMCloneTableui
```

Más información sobre cómo activar, configurar y forzar coredumps además de cómo extraer información útil:

https://linux-audit.com/understand-and-configure-core-dumps-work-on-linux/#:~:text=The%20default%20path%20where%20core,%2Flib%2Fsystemd%2Fcoredump

Videotutorial muy recomendable mostrando el uso de la herramienta **crash** para obtención interesante de ficheros core dump: https://www.youtube.com/watch?v=cxCYwN8jOHU

```
This program has absolutely no warranty.  Enter "help warranty" for details.

GNU gdb (GDB) 7.3.1
Copyright (C) 2011 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-unknown-linux-gnu"...

      KERNEL: /usr/lib/debug/lib/modules/2.6.32-431.11.2.el6.x86_64/vmlinux
    DUMPFILE: /var/crash/127.0.0.1-2014-03-24-06:53:29/vmcore  [PARTIAL DUMP]
        CPUS: 1
        DATE: Mon Mar 24 06:53:24 2014
      UPTIME: 00:01:55
LOAD AVERAGE: 0.16, 0.11, 0.04
       TASKS: 224
    NODENAME: centosbase.local
     RELEASE: 2.6.32-431.11.2.el6.x86_64
     VERSION: #1 SMP Tue Mar 25 19:59:55 UTC 2014
     MACHINE: x86_64  (2792 Mhz)
      MEMORY: 1 GB
       PANIC: "Oops: 0002 [#1] SMP " (check log for details)
         PID: 8796
     COMMAND: "bash"
        TASK: ffff880037f33540  [THREAD_INFO: ffff88002eb34000]
         CPU: 0
       STATE: TASK_RUNNING (PANIC)

crash>
```

```
  8615      1   0  ffff88003bdcd500  IN   0.2  188092   1688  automount
  8623      1   0  ffff88003b592080  IN   0.0    6280    304  mcelog
  8635      1   0  ffff88003b5beae0  IN   0.1   66608   1236  sshd
  8711      1   0  ffff88003c179540  IN   0.3   81280   3408  master
  8717   8711   0  ffff88003cc14ae0  IN   0.3   81360   3380  pickup
  8718   8711   0  ffff88003bdccaa0  IN   0.3   81428   3424  qmgr
  8735      1   0  ffff88003dcd0080  IN   0.1  110320   1000  abrtd
  8743      1   0  ffff88003df66ae0  IN   0.1  117300   1384  crond
  8754      1   0  ffff88003b593540  IN   0.0   21540    480  atd
  8766      1   0  ffff88003b5b2aa0  IN   0.1   62340    596  certmonger
  8779      1   0  ffff88003b5b2040  IN   0.1    4064    536  mingetty
  8781      1   0  ffff880037d90aa0  IN   0.1    4064    536  mingetty
  8783      1   0  ffff88003b592ae0  IN   0.1    4064    536  mingetty
  8785    424   0  ffff88003b5c8aa0  IN   0.2   12388   2608  udevd
  8786    424   0  ffff88003b5deaa0  IN   0.2   12388   2612  udevd
  8787      1   0  ffff88003dd4d540  IN   0.1    4064    532  mingetty
  8789      1   0  ffff880037662080  IN   0.1    4064    532  mingetty
  8791      1   0  ffff88003dd4cae0  IN   0.1    4064    532  mingetty
  8792   8635   0  ffff880037f32080  IN   0.4  100360   4052  sshd
> 8796   8792   0  ffff880037f33540  RU   0.2  108340   1796  bash
crash> files
PID: 8796   TASK: ffff880037f33540  CPU: 0   COMMAND: "bash"
ROOT: /    CWD: /root
 FD       FILE            DENTRY           INODE       TYPE PATH
  0 ffff88003dd04b00 ffff880030ef9140 ffff880030eef418 CHR  /dev/pts/0
  1 ffff88002eabe0c0 ffff880030c34300 ffff880030ee2598 REG  /proc/sysrq-trigger
  2 ffff88003dd04b00 ffff880030ef9140 ffff880030eef418 CHR  /dev/pts/0
 10 ffff88003dd04b00 ffff880030ef9140 ffff880030eef418 CHR  /dev/pts/0
255 ffff88003dd04b00 ffff880030ef9140 ffff880030eef418 CHR  /dev/pts/0
crash>
```

Ejemplo de salida de **crash** sobre un fichero core dump

7.2.2. Análisis de ficheros de paginación.

La mayoría de sistemas operativos modernos usan un espacio de intercambio (también conocido como archivo de paginación o memoria virtual). A este espacio también se le suele llamar swap, ya que permite al sistema hacer swaping o intercambios de memoria entre la RAM y el disco.

El área de intercambio se trata por tanto de una zona del disco (fichero o partición) que se usa para guardar las imágenes de los procesos que no han de mantenerse en memoria física mientras estos no están en ejecución. En el caso de Linux, la implementación de este espacio suele hacerse mediante una partición de disco aunque también permite el uso de ficheros de intercambio.

De una forma muy simplificada, se puede considerar que la memoria de intercambio hace las veces de una RAM adicional.

Gestión del espacio de intercambio en Linux

Para crear un área de intercambio, se usa la orden **mkswap nombre**, donde **nombre** puede ser un fichero o la ruta a un dispositivo de disco (por ejemplo /dev/hda1). Esto prepara el dispositivo y le pone la cabecera apropiada.

Si lo que se está usando es un fichero swap, el contenido puede ser cualquiera (se destruirá con el uso), y lo que importa es el tamaño. Pero, además, **mkswap** tiene un requisito especial: el fichero no ha de tener agujeros. Quiere decir que los bytes del fichero han de estar realmente en el disco. Esto no pasa siempre: si se usa **cp /dev/zero** nuevo, el sistema de ficheros probablemente vea que se trata de un fichero disperso (tiene regiones muy grandes todas con nulos), y no lo grabará realmente en el disco. Al usar algo como **dd if=/dev/zero of=nuevo bs=1024 count=65536** se le fuerza a que escriba todos los bloques, sin agujeros.

El dispositivo de intercambio se activa con la orden **swapon nombre** y se desactiva con **swapoff nombre**. En el fichero **/proc/swaps** se muestra una lista de los dispositivos de intercambio activos, y en **/etc/fstab** se suelen añadir las particiones swap que se activan al inicio del sistema.

Ejemplo práctico sobre la creación de un fichero swap en Linux: https://linuxize.com/post/how-to-add-swap-space-on-ubuntu-18-04/

Con el programa **free** se puede ver, además de cuánta RAM queda libre, el tamaño total de las áreas de intercambio, cuán de llenas están, y cuánta memoria (RAM) se está usando como caché.

```
admin1@mru8:~$ free -hm
              total        used        free      shared  buff/cache   available
Mem:           962M        219M        113M        1.2M        628M        568M
Swap:          1.9G        268K        1.9G
```

Ejemplo de salida del comando free

Igual que cualquier otro binario, la memoria swap puede ser leída como si se tratara de cualquier otro fichero, permitiendo incluso llegar a leer porciones de texto en claro con herramientas como **strings**, que busca e imprime automáticamente cadenas ASCII detectadas de un fichero binario o flujo o **hd**, que además de ASCII permite trabajar como editor decimal, hexadecimal y octal.

```
admin1@mru8: ~
admin1@mru8:~$ hd core.2009 | head -n 50
00000000  7f 45 4c 46 02 01 01 00  00 00 00 00 00 00 00 00  |.ELF............|
00000010  04 00 3e 00 01 00 00 00  00 00 00 00 00 00 00 00  |..>.............|
00000020  40 00 00 00 00 00 00 00  48 aa 9f 01 00 00 00 00  |@.......H.......|
00000030  00 00 00 00 40 00 38 00  8e 00 40 00 90 00 8f 00  |....@.8...@.....|
00000040  04 00 00 00 04 00 00 00  50 1f 00 00 00 00 00 00  |........P.......|
00000050  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
00000060  e0 2a 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |.*..............|
00000070  01 00 00 00 00 00 00 00  01 00 00 00 04 00 00 00  |................|
00000080  30 4a 00 00 00 00 00 00  00 50 c2 50 32 56 00 00  |0J.......P.P2V..|
00000090  00 00 00 00 00 00 00 00  00 30 00 00 00 00 00 00  |.........0......|
000000a0  00 30 00 00 00 00 00 00  01 00 00 00 00 00 00 00  |.0..............|
000000b0  01 00 00 00 06 00 00 00  30 7a 00 00 00 00 00 00  |........0z......|
000000c0  00 80 c2 50 32 56 00 00  00 00 00 00 00 00 00 00  |...P2V..........|
000000d0  00 10 00 00 00 00 00 00  00 10 00 00 00 00 00 00  |................|
000000e0  01 00 00 00 00 00 00 00  01 00 00 00 06 00 00 00  |................|
000000f0  30 8a 00 00 00 00 00 00  00 90 c2 50 32 56 00 00  |0..........P2V..|
00000100  00 00 00 00 00 00 00 00  00 90 00 00 00 00 00 00  |................|
00000110  00 90 00 00 00 00 00 00  01 00 00 00 00 00 00 00  |................|
00000120  01 00 00 00 06 00 00 00  30 1a 01 00 00 00 00 00  |........0.......|
00000130  00 30 50 52 32 56 00 00  00 00 00 00 00 00 00 00  |.0PR2V..........|
00000140  00 00 03 00 00 00 00 00  00 00 03 00 00 00 00 00  |................|
00000150  01 00 00 00 00 00 00 00  01 00 00 00 04 00 00 00  |................|
00000160  30 1a 04 00 00 00 00 00  00 00 b2 d4 ee 7f 00 00  |0...............|
00000170  00 00 00 00 00 00 00 00  00 30 00 00 00 00 00 00  |.........0......|
```

Ejemplo de salida de la herramienta hd aplicada sobre un fichero core dump

El uso de este tipo de herramientas es especialmente útil a la hora de depurar máquinas y extraer información de ficheros de la máquina. Por ejemplo en un análisis forense pueden utilizarse para extraer infomación importante (IPs, claves publicas/privadas, etc.) de ficheros como swapfiles, coredumps, pagefiles, etc. De hecho, debido a la cantidad de información sensible que suelen contener estos ficheros, el uso de estas herramientas y procedimientos son habituales también entre atacantes.

Algunas herramientas automatizadas para la extracción de información importante de los ficheros y particiones swap:

Swapdigger:
https://github.com/sevagas/swap_digger

Volatility: Realmente se trata suite muy amplia y potente para la obtención y análisis forense de memoria en máquinas tanto UNIX/Linux como Windows, pero también puede ser utilizada para trabajar con swap

https://github.com/volatilityfoundation/volatility/wiki/Linux

7.2.3. System Hang (Congelados del sistema)

Si una máquina se ha quedado congelada y antes de apagarla, queremos almacenar toda la información posible de sus últimos momentos de ejecución para poder analizarlos, tenemos algunas técnicas para detener la máquina de forma "controlada" y salvar su estado:

- Usar la Magic SysReq key o REISUB
- Usar NMI (Non-Maskable Interrupt) para forzar un Linux Kernel Crash Dump

7.2.4. El directorio virtual /proc

Este directorio virtual contiene toda la información relacionada con los procesos que se están ejecutando en el kernel de Linux. En la siguiente unidad se hará un recorrido exhaustivo por los distintos contenidos de este directorio.

7.2.5. Añadir printk's al kernel

A pesar de que tanto el kernel como los módulos están escritos en C, no podemos usar la instrucción printf de C porque esta se ejecuta en modo usuario y esto nos sacaría del modo kernel, por lo que se deberán utilizar funciones básicas del kernel como printk. Esta opción es válida tanto para módulos del kernel como para el propio kernel, aunque implica que una vez modificado para añadir los prints, sea compilado e instalado en la máquina

8. MÓDULOS PAM (PLUGGABLE AUTHENTICATION MODULES)

Los módulos PAM son un framework de componentes comunes dedicados a aportar autenticación y medidas de seguridad en los sistemas Linux. La información que arrojan dichos módulos (logs, systemd, etc) son especialmente útiles a la hora de detectar y analizar ataques e intentos de acceso a la máquina, como por ejemplo ejecución de fuerza bruta sobre el servicio SSH de la máquina con el objetivo de realizar un login con alguno de los usuarios por defecto típicos (root, login, admin, etc.)

8.1. ¿QUÉ ES UN MÓDULO PAM?

Un módulo PAM es un programa que se conecta al Kernel para ayudarle a éste a decidir si un determinado usuario puede o no acceder a una determinada operación.

El sistema PAM utiliza una arquitectura modular, donde cada módulo puede ser conectado o desconectado según la necesidad, ofreciendo al administrador del sistema una gran flexibilidad

para establecer políticas de autenticación para el sistema.

8.2. ALGUNAS DE LAS CARACTERÍSTICAS QUE OFRECE PAM

Provee un esquema de autenticación común que puede ser usado en una amplia variedad de aplicaciones.

- Aporta flexibilidad y control sobre la autenticación orientados a administradores de sistema y desarrolladores de aplicaciones.
- Provee una librería unificada junto la documentación correspondiente para permitir a desarrolladores escribir nuevos módulos PAM fácilmente.

8.3. CONFIGURACIÓN DE LOS MÓDULOS PAM

Relacionados con los módulos PAM podemos encontrar los siguientes ficheros de configuración:

- **/etc/pam.conf**: Fichero global de configuración, en él se declara la configuración de los módulos que no han creado su correspondiente fichero en el path /etc/pam.d/… Lo recomendado es utilizar la siguiente opción.
- **/etc/pam.d/[NOMBRE_SERVICIO]**: Por cada módulo PAM, se debe crear un fichero de configuración asociado.

Formato de los ficheros de configuración de los módulos PAM

En cada línea de los ficheros de configuración, nos encontramos el siguiente patrón, el cual aplica una configuración sobre un módulo determinado.

```
module_interface      control_flag      module_name      module_arguments
```

- **module_interface**: Soporta 4 valores posibles, en función del propósito del módulo. En función de la interfaz, el módulo recibirá los datos en un formato determinado orientado al propósito específico.
    - **auth:** Este tipo de interfaz es orientado a módulos que ofrecen mecanismos de autenticación.
    - **account**: Este tipo de interfaz es orientado a módulos que se encargan de verificar si el acceso está permitido.
    - **password**: Esta interfaz es orientada a módulos orientados al cambio de passwords.
    - **session**: Interfaz orientada a módulos dedicados a la gestión de configuración y gestión de sesiones de usuario.

- **control_flag**: Cada módulo genera como resultado un estado satisfactorio o fallido, las control_flag indican al sistema PAM como tratar con este resultado. Los módulos pueden ser apilados siguiendo un determinado orden y las control_flag determinan como de importantes son los resultados de cada módulo para el objetivo global de autenticación del usuario en el dispositivo.
  Algunos de los valores permitidos:
  - **required**: El resultado del módulo debe ser satisfactorio para continuar con el proceso. Si el módulo falla, la notificación no llega al usuario hasta que no se han evaluado el resto de módulos.
  - **requisite**: El resultado del módulo debe ser satisfactorio para continuar. Si el módulo falla, la notificación al usuario llega inmediatamente.
  - **sufficient**: El resultado del módulo es ignorado si falla, sin embargo, si el módulo se ejecuta correctamente y no existen otros módulos que tengan la etiqueta required, el usuario puede acceder al sistema.
  - **optional**: El resultado del módulo es ignorado salvo en el caso de que no existan más módulos sobre la interfaz.
  - **include**: Esta orden es diferente de las anteriores, permite importar otros ficheros de configuración.

  Existen algunos otros valores menos frecuentes que se utilizan siguiendo el formato clave=valor, la lista de todas las opciones disponibles se puede encontrar aquí.
- **module_name**: Nombre del módulo PAM.
- **module_arguments**: Lista de argumentos utilizados por el módulo PAM durante su ejecución. Se debe especificar una lista separada por comas donde cada elemento de esa lista sigue el formato clave=valor.

Bibliografía

- Tutorial para la obtención y análisis forense de la memoria de una máquina utilizando Volatility: https://www.luisguillen.com/posts/2018/01/analisis-forense-volatility-virtualbox-ubuntu/
- http://web.mit.edu/rhel-doc/3/rhel-sag-es-3/ch-kernel-modules.html